

Rendu individuel

Elyess Rjafellah
CI/CD & automatisation

Auteur	Elyess Rjafellah
Rôle	CI/CD & automatisation
Classe / Promo	M2 DO C — 2025-2026
Période	Janvier – Juin 2026
Projet	Plateforme de gestion d'un cabinet d'audioprothèse
Domaine	Chaîne GitHub Actions, état distant, pipeline auto-réparant

SUP DE VINCI — Expert en Systèmes d'Information — Mastère DevOps — Classe **M2 DO C** — Promotion 2025-2026. Document réalisé dans le cadre du projet d'étude de fin d'année.

1. Contexte et rôle dans le projet

Ma mission a porté sur le cœur de la démarche DevOps : la chaîne d'intégration et de déploiement continu. L'objectif fixé collectivement était ambitieux : obtenir un déploiement « en un clic », où l'intégralité du cycle de vie — provisionnement de l'infrastructure, construction et analyse des images, déploiement de l'application, configuration du stockage de sauvegarde — s'exécute automatiquement, sans aucune intervention manuelle, et se rétablit seul en cas d'aléa.

Ce rôle est transversal par nature : la chaîne que j'ai construite orchestre le travail de tous les autres membres, en assemblant l'infrastructure de Zaafer, le chart de déploiement d'Anis, la supervision et les scans de sécurité d'Adame. J'ai donc dû comprendre chaque brique pour l'intégrer au bon moment et dans le bon ordre au sein du pipeline.

2. Ma contribution détaillée

2.1 Architecture de la chaîne GitHub Actions

J'ai conçu plusieurs workflows complémentaires. Le workflow d'intégration valide chaque modification par l'analyse statique du code et l'exécution des tests, côté backend comme côté frontend. Le workflow de sécurité exécute les analyses de vulnérabilités. Le workflow de déploiement orchestre l'ensemble de la mise en production. Enfin, un workflow de sauvegarde, planifié quotidiennement, déclenche la réplication des données vers le site on-premise.

Cette séparation en workflows spécialisés répond à un souci de clarté et d'efficacité : les vérifications rapides (tests, analyses) s'exécutent à chaque proposition de modification, tandis que les opérations lourdes et coûteuses (provisionnement, déploiement) ne sont déclenchées que de manière maîtrisée, sur la branche principale ou manuellement.

2.2 Automatisation du provisionnement et de l'état

J'ai automatisé la préparation de l'état distant de Terraform : le workflow crée si nécessaire le compte de stockage dédié, puis initialise Terraform sur cet état avant d'appliquer l'infrastructure. Cette étape, souvent laissée manuelle dans les projets, a été entièrement scriptée pour que rien ne doive être préparé à l'avance.

L'authentification à Azure au sein de la chaîne s'appuie sur des identifiants stockés dans les secrets chiffrés de GitHub, jamais exposés dans les journaux. J'ai veillé à masquer explicitement les valeurs sensibles (chaînes de connexion, mots de passe) dans les traces d'exécution, afin qu'elles n'apparaissent jamais en clair, même en cas de débogage.

2.3 Orchestration et ordonnancement des étapes

Le déploiement complet enchaîne, dans un ordre précis, la connexion à Azure, l'application Terraform, la récupération des informations produites (adresses, identifiants), la construction et la publication des images, leur analyse de sécurité, l'installation des briques du cluster puis le déploiement de l'application et la configuration du site on-premise. J'ai dû gérer finement les dépendances : par exemple, m'assurer que les définitions de ressources nécessaires à la supervision sont installées **avant** l'application qui les référence, sous peine d'échec.

J'ai également optimisé les temps d'exécution en supprimant des attentes bloquantes inutiles : plutôt que d'attendre que chaque composant de supervision soit totalement prêt, la chaîne applique les manifestes et laisse les composants converger en arrière-plan, ce qui a réduit la durée du déploiement de façon significative.

2.4 Fiabilisation et auto-réparation

La partie la plus exigeante de mon travail a consisté à rendre le pipeline robuste face aux états intermédiaires. Un déploiement interrompu pouvait laisser un verrou sur l'état de l'infrastructure, ou une livraison Helm figée dans un état instable empêchant toute reprise. J'ai introduit des mécanismes qui détectent ces situations et les corrigent automatiquement : libération du verrou résiduel, nettoyage d'une livraison bloquée avant réinstallation.

Grâce à ces garde-fous, la chaîne est devenue **auto-réparante** : elle se remet d'elle-même d'un incident sans qu'un opérateur ait à intervenir sur l'infrastructure, ce qui est une propriété essentielle pour une exploitation sereine et conforme à l'esprit DevOps.

3. Choix techniques et justifications

Le cahier des charges évoquait GitLab CI ; nous avons retenu GitHub Actions, le dépôt étant hébergé sur GitHub. Le principe reste identique — build, test, analyse, déploiement orchestré — et GitHub Actions offre une intégration native avec le dépôt, un large catalogue d'actions réutilisables et une gestion des secrets intégrée. Ce choix a donc été pragmatique et sans perte fonctionnelle.

J'ai privilégié un déploiement piloté par la chaîne (approche « push ») plutôt qu'un outil de déploiement continu dédié, afin de rester simple et économe pour un MVP. Cette décision est réversible : j'ai identifié le passage à une approche déclarative de type GitOps comme évolution naturelle une fois le projet stabilisé.

4. Difficultés rencontrées et solutions apportées

J'ai été confronté à une série d'échecs typiques d'une première mise en production : une action dont la version n'existait pas, un conflit de création de ressource déjà existante, un verrou d'état résiduel, une livraison bloquée, un mauvais ordonnancement des dépendances. Plutôt que de les contourner au cas par cas, j'ai traité chaque incident à la racine, en ajoutant le correctif durable correspondant dans le pipeline.

Cette approche m'a beaucoup appris : un pipeline robuste ne se conçoit pas d'un seul jet, il se durcit par itérations successives face aux situations réelles. Le résultat est une chaîne qui, aujourd'hui, se déroule de bout en bout de façon fiable et reproductible.

5. Perspectives d'évolution

La première évolution que je viserais est l'adoption du GitOps, avec un outil tel qu'ArgoCD : l'état souhaité du cluster serait décrit dans le dépôt et un opérateur se chargerait de faire converger le cluster vers cet état, offrant traçabilité et retours arrière simplifiés.

J'introduirais ensuite une authentification fédérée OIDC pour supprimer les secrets de longue durée, ainsi que des environnements distincts — développement, pré-production, production — avec une promotion contrôlée des versions. Enfin, j'ajouterais des tests de bout en bout exécutés automatiquement après chaque déploiement, afin de valider le bon fonctionnement de l'ensemble avant d'ouvrir le service aux utilisateurs.

6. Analyse critique des limites

Regrouper le provisionnement et le déploiement dans un même workflow est pratique pour la démonstration, mais allonge la durée d'exécution ; un découpage en jobs parallélisés, voire en pipelines distincts pour l'infrastructure et l'application, serait plus efficace à l'échelle. L'authentification par identifiant et mot de passe, quant à elle, impose un compte sans double authentification, ce qui n'est pas acceptable en production.

Par ailleurs, l'absence d'environnement de pré-production fait que les essais se déroulent directement sur l'unique cluster, et un déclenchement automatique sur chaque envoi de code pourrait lancer des déploiements coûteux : un garde-fou explicite (validation manuelle, restriction de branche) mériterait d'être renforcé.

7. Annexe — documentation utilisateur (mon périmètre)

7.1 Lancer un déploiement

Depuis l'onglet Actions du dépôt, l'utilisateur sélectionne le workflow de déploiement et l'exécute en choisissant l'action de déploiement. La chaîne se charge de tout ; l'adresse d'accès à l'application est affichée à la dernière étape. Un déploiement peut aussi être déclenché automatiquement par un envoi de code sur la branche principale.

7.2 Détruire l'infrastructure

Le même workflow, exécuté avec l'action de destruction, supprime l'ensemble des ressources et arrête la facturation. L'opération est sûre et reproductible : un déploiement ultérieur reconstruit l'environnement à l'identique.

7.3 Sauvegarder et restaurer

Le workflow de sauvegarde s'exécute automatiquement chaque jour, et peut être déclenché manuellement en mode sauvegarde ou restauration. Les secrets nécessaires à ces workflows sont décrits dans la documentation d'installation du dépôt.

8. Analyse personnelle

Défis rencontrés

Mon principal défi a été de transformer une succession de scripts en une chaîne réellement fiable, capable d'absorber les aléas sans intervention humaine. Chaque échec rencontré était une énigme à résoudre à la racine, ce qui a demandé rigueur et persévérance.

Forces

Je retiens ma rigueur dans l'ordonnancement et l'idempotence des étapes, ainsi que ma capacité à diagnostiquer rapidement l'origine d'un échec de pipeline à partir des journaux d'exécution et à y apporter un correctif durable plutôt qu'un contournement ponctuel.

Faiblesses

Mes workflows sont encore trop monolithiques et gagneraient à être modularisés en briques réutilisables. Par ailleurs, j'ai concentré mes efforts sur le fonctionnement au détriment, parfois, de la sécurisation de la chaîne elle-même (authentification), que je dois approfondir.

Compétences développées

J'ai gagné en maîtrise sur GitHub Actions (workflows, secrets, déclencheurs, gestion de la concurrence), sur l'automatisation de Terraform et sur le diagnostic et la résilience des pipelines. J'ai surtout compris ce qui distingue une automatisation fragile d'une chaîne réellement industrielle.

Axes d'amélioration personnels

Je souhaite modulariser mes pipelines à l'aide de workflows réutilisables, mettre en place une véritable stratégie multi-environnements et adopter le GitOps, afin de tendre vers des pratiques de livraison continue de niveau professionnel.